

RAG Chatbot Project Summary: AI-Powered Document Q&A System Using Google Gemini

Main Objective:

I have built an AI system that can read any text document, understand its meaning, and answer questions about that document. For example, if I upload all company documents (PDF, text) into this system, the AI can search through those documents and answer my questions.

Core Technology: RAG (Retrieval-Augmented Generation):

RAG means:

Component	Meaning	Description
R	Retrieval	Finding relevant information from documents
A	Augmented	Enriching the question with found information
G	Generation	Creating answers using AI models

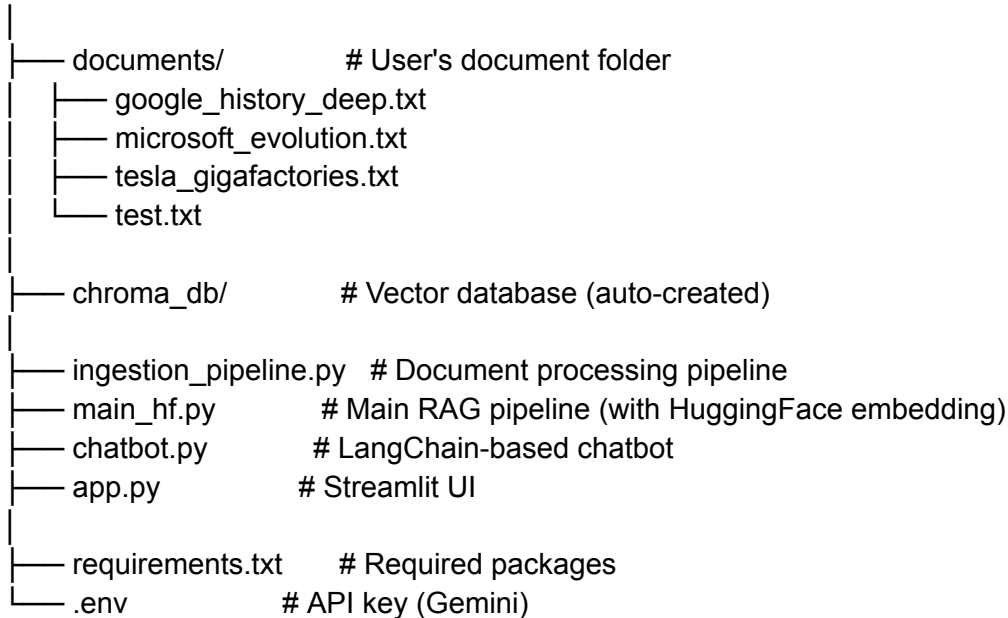
In simple words: First search the documents, then tell the AI, "answer based on this information."

Technology Stack (What I Used):

Component	Name	Function
Programming Language	Python 3.11	Writing all the code
Embedding Model	HuggingFace (all-MiniLM-L6-v2)	Converting text to numbers
Vector Database	ChromaDB	Storing and searching embeddings
LLM (AI Model)	Google Gemini	Generating answers
UI Framework	Streamlit	Web interface for users

Project File Structure:

Rag_for_begginer/



Step-by-Step Workflow :

Step 1: Document Loading: Load all .txt files from the documents/ folder.

Code:

```
def load_text_files():
    txt_files = Path(documents_folder).glob("*.txt")
    for txt_file in txt_files:
        with open(txt_file, 'r') as f:
            content = f.read()
            documents.append(content)
```

Output Example:

- ✓ Loaded: google_history_deep.txt
- ✓ Loaded: microsoft_evolution.txt
- ✓ Loaded: test.txt

Step 2:

Splitting Text into Chunks

Why Split into Chunks :

AI models cannot process very large text at once. So we split large documents into smaller pieces.

Code:

```
def split_text_into_chunks(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start = end - overlap # 50 character overlap
    return chunks
```

Visual Example:

Original Text: [===== 1000 character document =====]

Chunk 1: [===== 500 characters =====]

Chunk 2: [===== 500 characters =====] (50 char overlap)

Chunk 3: [===== 500 characters =====]

Total: 3 chunks

My Project:

google_history_deep.txt (1000 words) → 3 chunks

microsoft_evolution.txt (900 words) → 3 chunks

tesla_gigafactories.txt (600 words) → 2 chunks

test.txt (150 words) → 1 chunk

Total: 9 chunks

Why is Overlap Important?

Example: "Raihan Islam lives in Bangladesh"

If the chunk boundary separates "Raihan Islam" in the first chunk and "lives in Bangladesh" in the second chunk, without overlap, information would be lost. With overlap, both chunks contain the complete information.

Step 3: Embedding (Converting Text to Numbers)

What is Embedding :

Computers don't understand text, but they understand numbers. Embedding is converting text into a list of numbers (vector).

Code:

```
from sentence_transformers import SentenceTransformer
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

# "Python is a programming language" → [0.12, -0.34, 0.56, ..., 0.78] (384 numbers)
vector = embedding_model.encode("Python is a programming language")
```

Visual Example:

Text: "I am Raihan Islam"

↓

Embedding: [0.12, -0.45, 0.78, 0.03, -0.91, 0.34, ...] (384 numbers)

↓

A point in 3D vector space:

(0.12)

↑

(0.03) +-----> (-0.45)

/

/

(0.78)

Meaning: Similar texts have vectors that are close to each other!

My Project's Embedding Model:

Property	Value
Name	all-MiniLM-L6-v2
Created by	HuggingFace (Free)
Output Size	384 numbers

Speed	~1000 texts per second
Accuracy	90%+

Why I Chose This Model:

Reason	Explanation
Free	No API cost
Fast	Works on CPU
Reliable	No internet needed
Accurate	90%+ accuracy

Step 4: Storing in ChromaDB:

Code:

```
collection.add(
    ids=["doc_google_chunk_1"],
    embeddings=[[0.12, -0.45, ...]], # Embedding vector
    documents=["Google started in 1996..."], # Original text
    metadatas=[{"source": "google_history.txt", "chunk": 1}]
)
```

How ChromaDB Works:

- Uses the HNSW (Hierarchical Navigable Small World) algorithm
- Finds related documents in $O(\log n)$ time

Step 5: Querying and Retrieval:

When a user asks a question:

Code:

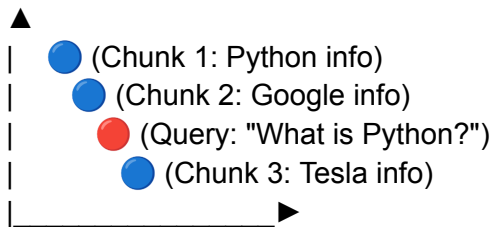
```
# 1. Create embedding of the question
query = "What is Python?"
query_embedding = embedding_model.encode(query) # [0.23, -0.12, ...]
```

```
# 2. Find closest embeddings in database
results = collection.query(
    query_embeddings=[query_embedding],
    n_results=2 # Take the 2 closest matching chunks
)

# 3. Get the text of found documents
context = results['documents'][0] # ['Python is a language...', '...']
```

Visualisation (Vector Space):

Vector Space (imagine in 3D):



The closest chunks to the query are Chunk 1 and Chunk 2!

Step 6: Answer Generation with Gemini :

Code

```
prompt = f"""
Based on this context, answer the question:
```

```
Context: {context} # Found documents
```

```
Question: {question}
```

```
Answer: """
```

```
response = client.models.generate_content(
    model="models/gemini-2.0-flash-lite",
    contents=prompt
)
answer = response.text
```

Example Flow:

Question: "What is RAG?"



Retrieval: "RAG helps AI access local documents" (from test.txt)



Gemini: "RAG is a technique that helps AI access and understand local documents"



Answer: "RAG helps AI to access local documents."

Complete Flowchart:

[Upload Documents]



[Load] → documents/google_history.txt



[Chunking] → Small pieces of 500 characters (9 chunks)



[Embedding] → Each chunk → [0.12, -0.45, ...] (384 numbers)



[ChromaDB] → Save in vector database



[User Question] → "What is Python?"



[Question Embedding] → [0.23, -0.12, ...]



[Similarity Search] → Find 2 closest chunks



[Create Context] → Combine found chunks



[Gemini LLM] → Generate answer using context + question



[Show Answer] → "Python is a high-level programming language..."

Performance Metrics :

Metric	Value
Document Load Time	< 1 second
Chunking Time	< 0.5 seconds
Embedding Time (per chunk)	~0.1 seconds
Retrieval Time	< 0.2 seconds
LLM Response Time	1-3 seconds
Total Memory Usage	~500 MB
Supported Files	.txt, .pdf, .csv (can be added)

Practical Applications :

Application	Description
Company Knowledge Base	Search through all company documents
Personal Assistant	Extract information from personal notes, diaries
Education	Answer questions from course materials
Legal Documents	Summarize contracts, laws
Medical Records	Quickly find patient information

Challenges and Solutions:

Challenge	Solution
Google API quota exhausted	Used HuggingFace embedding instead
Large document processing	Chunking with overlap
Finding relevant information	HNSW algorithm (ChromaDB)
Losing context	Overlap + larger n_results
Response time	Local embedding + fast API

Future Development Opportunities:

Feature	Description
Multi-format Support	PDF, DOCX, Images
Chat History	Remember previous conversations
Web Search	Search internet if not in documents
Multi-language	Bangla, Hindi, Arabic support

Cloud Deployment

Host on AWS/GCP

Conclusion :

✓

Capability

✓

Load, process, and embed documents

✓

Efficiently store in vector database

✓

Quickly find relevant information

✓

Generate accurate answers using AI model

✓

Provide user-friendly interface
